

```

clear all;
clc;
%% Target
z = [0:2:256]';
nz = length(z);

u_H = 6.0;          % Friction velocity [m/s]
z_H = 20;

u_target = u_H * (z/ z_H).^0.16;
u(1) = 0;  % Set surface velocity to zero

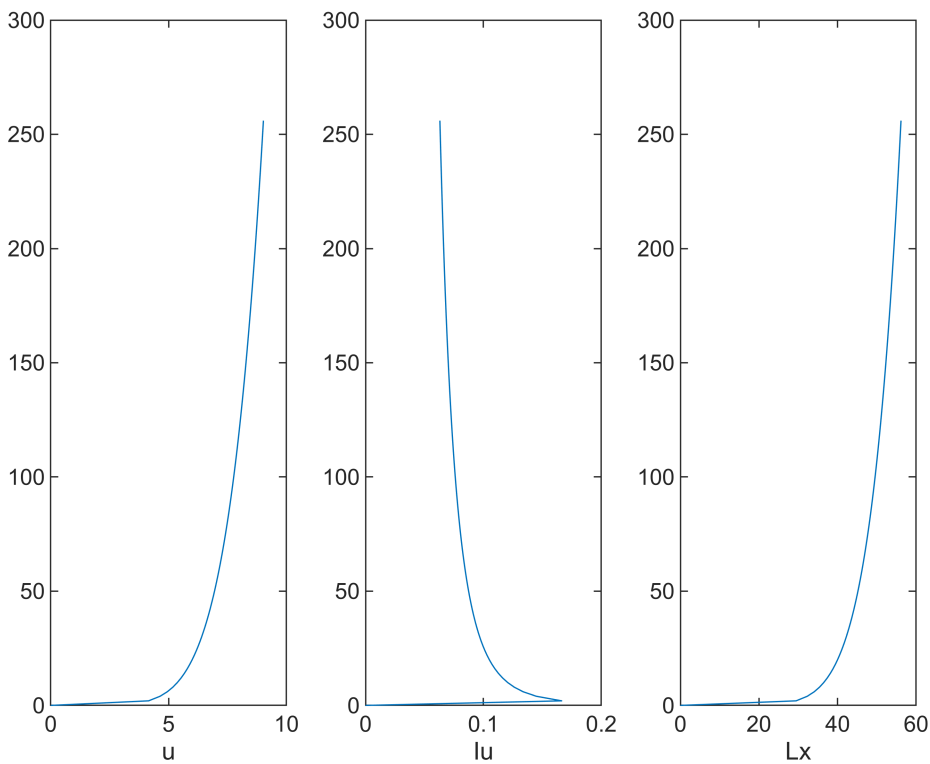
I_H = 0.105;
Iu_target = I_H*(z/z_H).^(-0.2);
Iu_target(~isfinite(Iu_target)) = 0;

Iv_target = 0.78* Iu_target;
Iw_target = 0.55* Iu_target;

L_H = 2.0 * z_H;
Lx_target = L_H * (z/z_H).^(0.133);
Ly_target = 0.3*Lx_target(floor(nz/2)) * ones(size(Lx_target));
Lz_target = 0.5*Lx_target(floor(nz/2)) * ones(size(Lx_target));

Lx_target = Lx_target*2/pi;
Ly_target = Ly_target*2/pi;
Lz_target = Lz_target*2/pi;
figure
subplot(1,3,1); plot(u_target, z); xlabel 'u'; ylabel 'z [m]'
subplot(1,3,2); plot(Iu_target, z); xlabel 'Iu'
subplot(1,3,3); plot(Lx_target, z); xlabel 'Lx'

```



```
%% Test turbulence mean and intensity
% load data
jtot = 32;
ktot = 128;
u_ins = load_driver('udriver_000.770',jtot, ktot);
```

```
Stored y points including halos: 34
Stored z points including halos: 130
Stored time records:          20001
File size:                   0.659 GB
Interior array size:
    32          128          20001
```

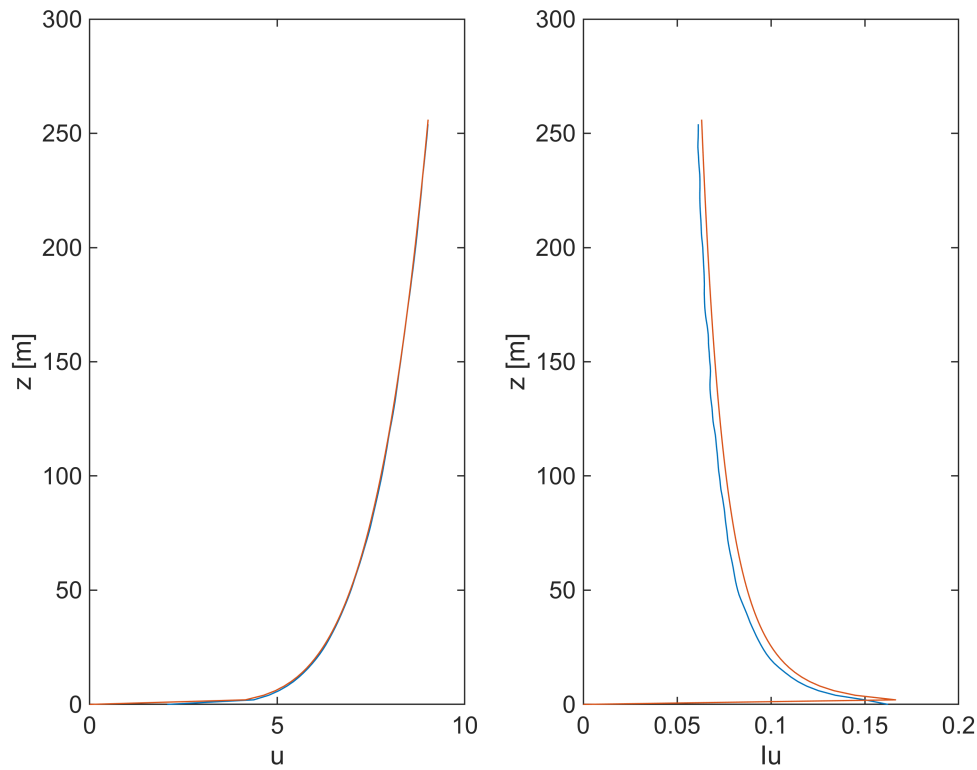
```
u_ins = u_ins(:,:,3000:end);
```

```
%% mean velocity and turbulent intensity
zt= 0:2:256-2;
```

```
Umean = mean(mean(u_ins, 1), 3);
uFluc = u_ins - Umean;
urms = sqrt(mean(mean(uFluc.^2, 1), 3));
Iu = urms ./ abs(Umean);
%
```

```
figure
subplot(1,2,1)
plot(Umean, zt, u_target, z); xlabel 'u'; ylabel 'z [m]'
```

```
subplot(1,2,2)
plot(Iu, zt, Iu_target, z);xlabel 'Iu'; ylabel 'z [m]'
```



```
%% Turbulent integral length Luu_x
u_load = u_ins;
dt = 1;
ny = size(u_load,1);
nz = size(u_load,2);
nt = size(u_load,3);

maxLag = floor((nt-1)/2);
tau = (0:maxLag)'*dt;
nfft = 2^nextpow2(2*nt-1);

meanU = zeros(nz,1);
Tuu = zeros(nz,1);
Luu = zeros(nz,1);
Ruu = zeros(maxLag+1,nz);

numberPairs = ny*(nt-(0:maxLag)');

for iz = 1:nz
    meanU(iz) = mean(u_load(:,iz,:), 'all');
    up = u_load(:,iz,:) - meanU(iz);
    varianceU = mean(up(:).^2);
```

```

% FFT temporal autocorrelation.
up_hat = fft(up,nfft,3);
autocovariance = real(ifft(abs(up_hat).^2,[],3));
autocovariance = squeeze(sum(autocovariance,1));
autocovariance = autocovariance(1:maxLag+1);

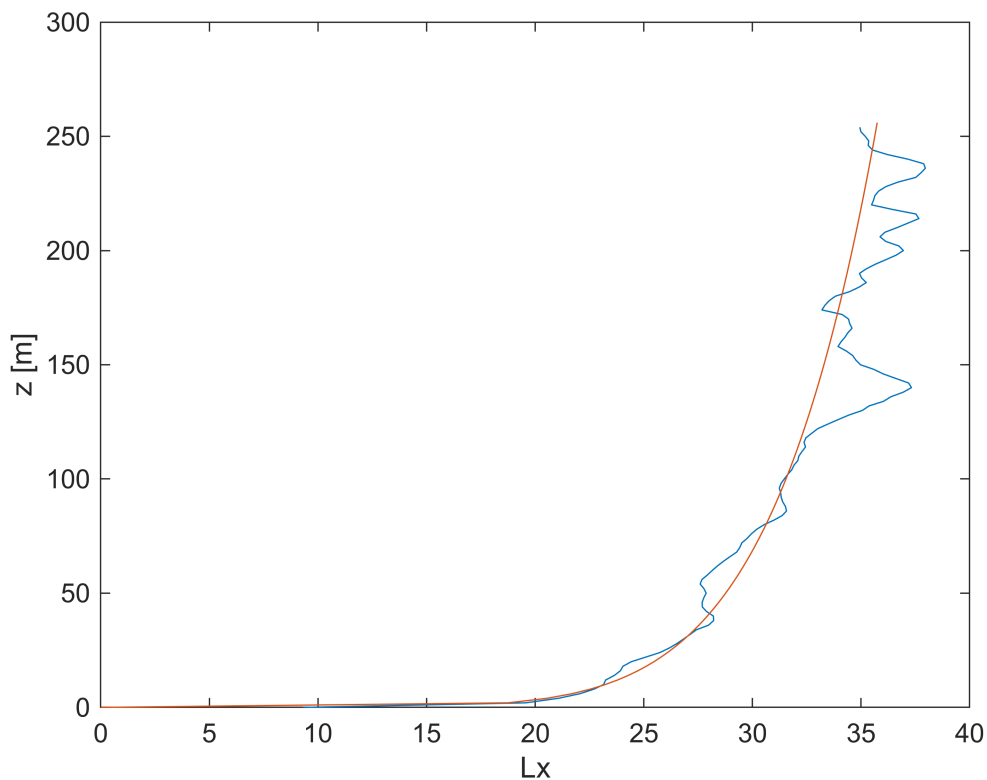
% Unbiased covariance: account for fewer pairs at large lag.
covariance = autocovariance./numberPairs;
Ruu(:,iz) = covariance/varianceU;
Ruu(1,iz) = 1;

firstZero = find(Ruu(2:end,iz) <= 0,1,'first');

if isempty(firstZero)
    Tuu(iz) = trapz(tau,Ruu(:,iz));
else
    Tuu(iz) = trapz(tau(1:firstZero), ...
                    Ruu(1:firstZero,iz));
end

Luu(iz) = abs(meanU(iz))*Tuu(iz);
end
figure
plot(Luu, zt); hold on
plot(Lx_target, z); xlabel 'Lx'; ylabel 'z [m]'

```



```

%% Luu(y)
dy = 2;
ny = size(u_ins,1);
nz = size(u_ins,2);
maxLag = floor((ny-1)/2);
deltaY = (0:maxLag)'*dy;

meanU = zeros(nz,1);
LuuY = zeros(nz,1);
RuuY = zeros(maxLag+1,nz);

for iz = 1:nz
    meanU(iz) = mean(u_ins(:,iz,:), 'all');
    up = u_ins(:,iz,:) - meanU(iz);
    varianceU = mean(up(:).^2);
    RuuY(1,iz) = 1;

    % Spatial autocorrelation in the y direction.
    for lag = 1:maxLag
        up1 = up(1:end-lag,:);
        up2 = up(1+lag:end,:);
        covariance = mean(up1(:).*up2(:));
        RuuY(lag+1,iz) = covariance/varianceU;
    end

    firstZero = find(RuuY(2:end,iz) <= 0,1, 'first');

    if isempty(firstZero)
        LuuY(iz) = trapz(deltaY, RuuY(:,iz));
        warning('RuuY did not cross zero at z index %d.',iz);
    else
        LuuY(iz) = trapz(deltaY(1:firstZero), ...
                        RuuY(1:firstZero,iz));
    end
end
end

```

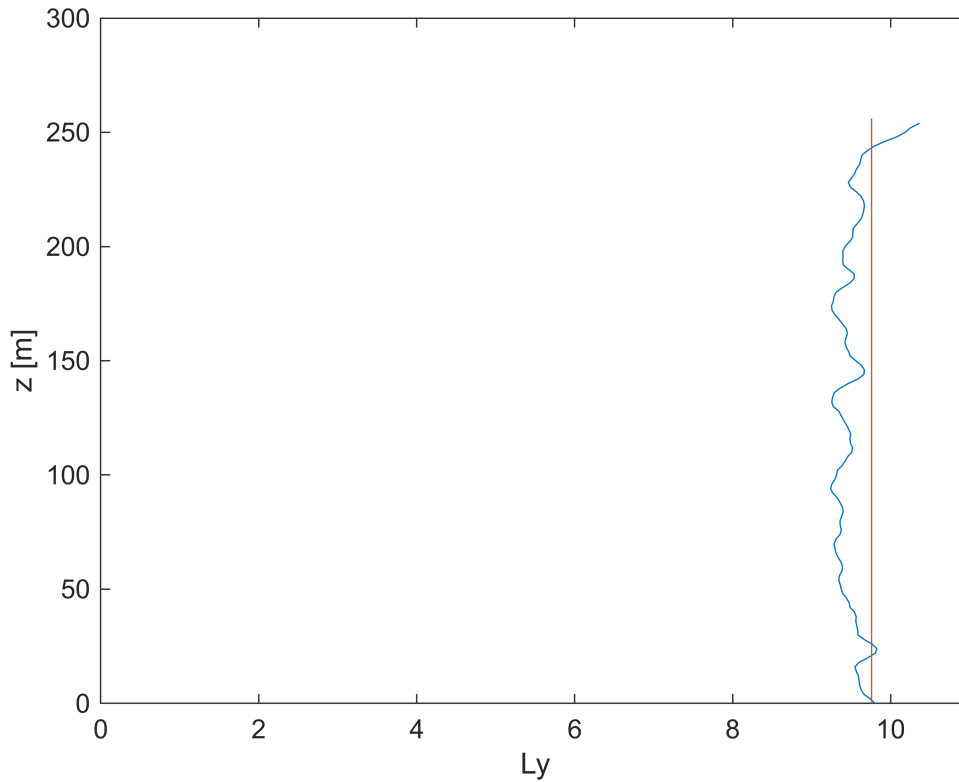
```

Warning: RuuY did not cross zero at z index 1.
Warning: RuuY did not cross zero at z index 11.
Warning: RuuY did not cross zero at z index 12.
Warning: RuuY did not cross zero at z index 13.
Warning: RuuY did not cross zero at z index 14.
Warning: RuuY did not cross zero at z index 94.
Warning: RuuY did not cross zero at z index 95.
Warning: RuuY did not cross zero at z index 102.
Warning: RuuY did not cross zero at z index 103.
Warning: RuuY did not cross zero at z index 104.
Warning: RuuY did not cross zero at z index 106.
Warning: RuuY did not cross zero at z index 107.
Warning: RuuY did not cross zero at z index 108.
Warning: RuuY did not cross zero at z index 109.
Warning: RuuY did not cross zero at z index 110.
Warning: RuuY did not cross zero at z index 111.
Warning: RuuY did not cross zero at z index 112.
Warning: RuuY did not cross zero at z index 123.

```

Warning: RuuY did not cross zero at z index 124.
Warning: RuuY did not cross zero at z index 125.
Warning: RuuY did not cross zero at z index 126.
Warning: RuuY did not cross zero at z index 127.
Warning: RuuY did not cross zero at z index 128.

```
figure
plot(LuuY,zt); hold on
plot(Ly_target,z); xlabel 'Ly'; ylabel 'z [m]'
xlim([0 11])
```



```
%% Luu(z)
dz = 2;
nz = size(u_ins,2);
maxLag = floor((nz-1)/2);
deltaZ = (0:maxLag)'*dz;

meanU = zeros(nz,1);
LuuZ = zeros(nz,1);
RuuZ = zeros(maxLag+1,nz);

for iz = 1:nz
    meanU(iz) = mean(u_ins(:,iz,:), 'all');
end
```

```

for iz = 1:nz
    up0 = u_ins(:,iz,:) - meanU(iz);
    variance0 = mean(up0(:).^2);
    RuuZ(1,iz) = 1;

    % Choose the vertical direction with sufficient available points.
    if iz + maxLag <= nz
        direction = 1;
    else
        direction = -1;
    end

    % Spatial autocorrelation in the z direction.
    for lag = 1:maxLag
        iz2 = iz + direction*lag;

        up1 = u_ins(:,iz2,:) - meanU(iz2);
        variance1 = mean(up1(:).^2);
        covariance = mean(up0(:).*up1(:));

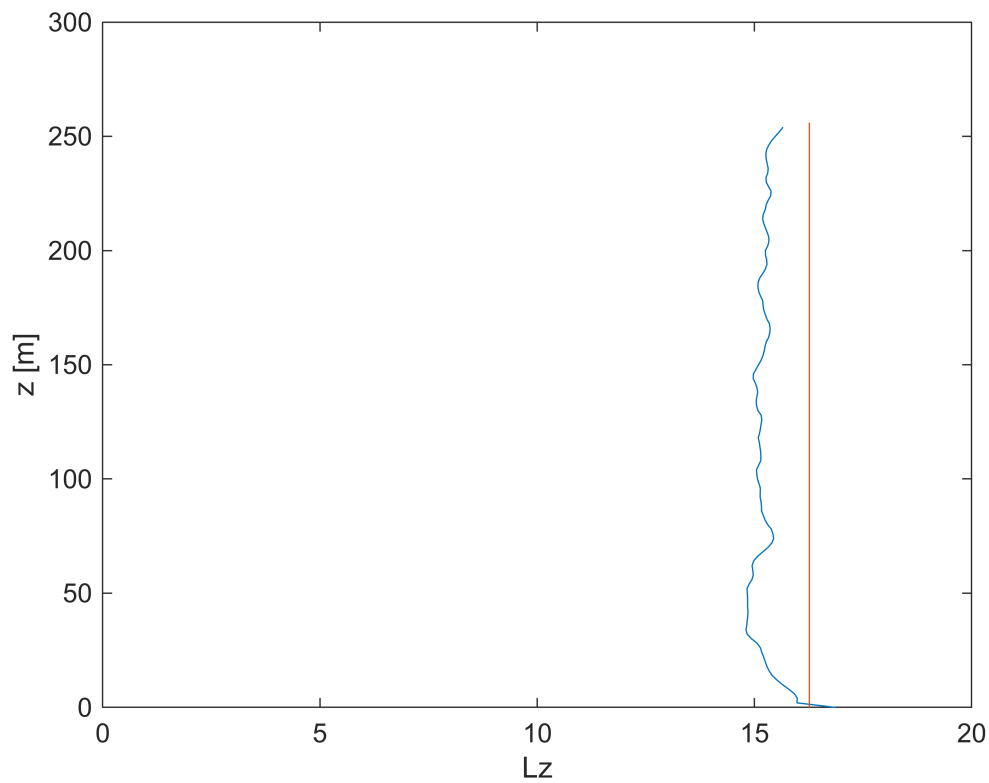
        RuuZ(lag+1,iz) = covariance / ...
            sqrt(variance0*variance1);
    end

    firstZero = find(RuuZ(2:end,iz) <= 0,1,'first');

    if isempty(firstZero)
        LuuZ(iz) = trapz(deltaZ,RuuZ(:,iz));
        warning('RuuZ did not cross zero at z index %d.',iz);
    else
        LuuZ(iz) = trapz(deltaZ(1:firstZero), ...
            RuuZ(1:firstZero,iz));
    end
end

figure
plot(LuuZ,z); hold on
plot(Lz_target,z);xlabel 'Lz'; ylabel 'z [m]'
xlim([0 20])

```



```

%% frequency spectrum
dt = 1;
fs = 1 / dt;

nfft = 512;
window = hann(nfft, 'periodic');
noverlap = nfft / 2;

nf = nfft/2 + 1;
Suu = zeros(nf, ktot);

for iz = 1:ktot
    Suu_sum = zeros(nf, 1);
    nValid = 0;

    for iy = 1:jtot
        ui = double(squeeze(u_ins(iy, iz, :)));

        ui = ui - mean(ui, 'omitnan');
        [Ptmp, f] = pwelch(ui, window, noverlap, ...
                           nfft, fs, 'onesided');

        Suu_sum = Suu_sum + Ptmp;
        nValid = nValid + 1;
    end
end

```



```

end

Suu(:, iz) = Suu_sum / nValid;
end

% variance_from_spectrum = trapz(f, Suu(:, iz));
% urms_from_spectrum = sqrt(variance_from_spectrum);
%
% ui = reshape(u_ins(:, iz, :), [], 1);
% urms_direct = std(ui, 1, 'omitnan');
iz = 20;
figure
loglog(f(2:end), Suu(2:end, iz), 'LineWidth', 1.5)
xlabel('f (Hz)')
ylabel('S_{uu}(f) [(m/s)^2/Hz]')
grid on

```

